UT4 OPTIMIZACIÓN Y DOCUMENTACIÓN DEL CÓDIGO

Vázquez García, Iván
Echeverría Gómez, Fernando

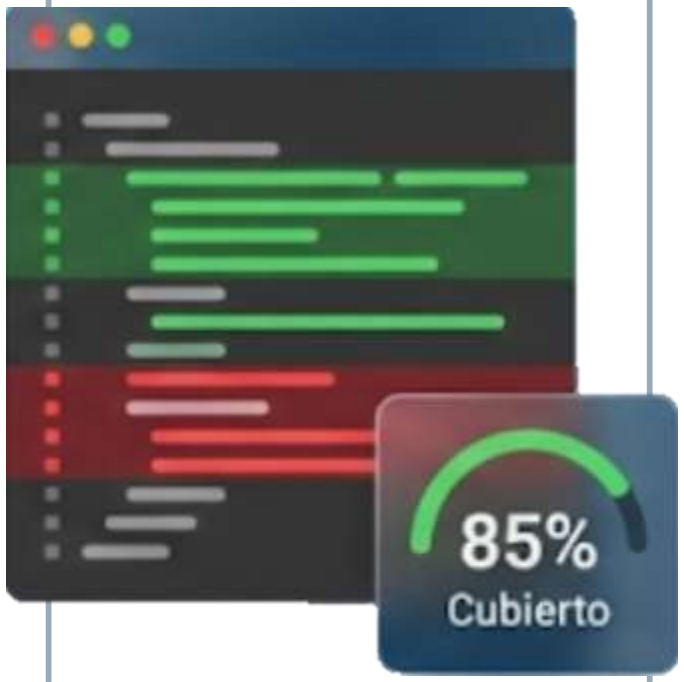
1. Refactorización

- ¿Qué es la refactorización?
- Objetivo principal: mejora del código sin cambiar su comportamiento.
- Beneficios: legibilidad, mantenibilidad, calidad del diseño.

1. La Refactorización

¿Qué es?

La refactorización se define formalmente como una técnica disciplinada de reestructuración del código existente que altera su estructura interna sin cambiar su comportamiento externo observable.



Objetivo principal

Su objetivo principal no es corregir errores ni añadir funcionalidades sino reducir la deuda técnica acumulada. Se trata de un proceso de limpieza y perfeccionamiento del diseño que permite que el software siga siendo modificable y sostenible a largo plazo evitando que el código se degrade con el tiempo.

1. Beneficios de la Calidad del Código

Calidad del Código

La aplicación correcta de la refactorización aporta beneficios directos a la rentabilidad del proyecto. Mejora drásticamente la legibilidad permitiendo que cualquier desarrollador entienda la lógica rápidamente sin necesidad de descifrar código complejo. Facilita la mantenibilidad futura reduciendo el tiempo necesario para implementar nuevos cambios. Además eleva la calidad del diseño.

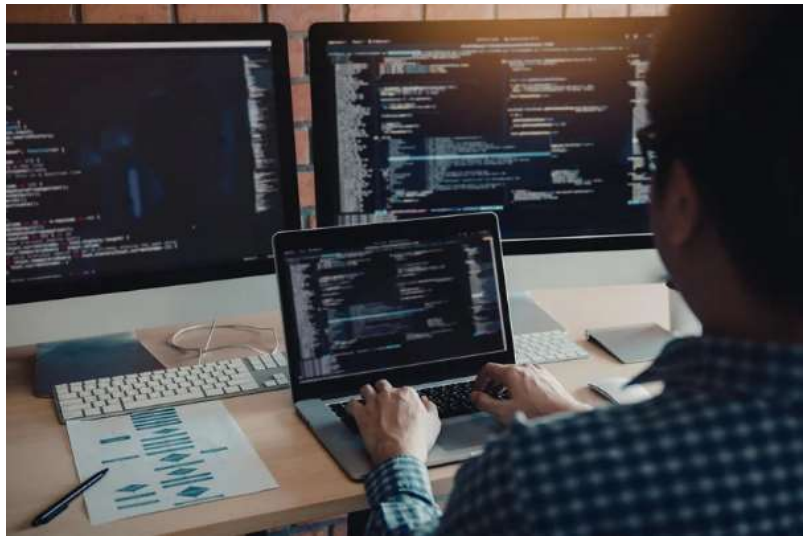
1.

Limitaciones Funcionalidad y Riesgos

Limitaciones de Funcionalidad

Es fundamental entender que la refactorización es un proceso invisible para el usuario final ya que estrictamente no añade ninguna funcionalidad nueva al sistema. Existe un riesgo técnico considerable si se realiza sin una red de seguridad adecuada pues se pueden introducir errores de regresión que rompan características que ya funcionaban.

Modificar código complejo que no se comprende totalmente sin el respaldo de pruebas automáticas es una práctica peligrosa que debe evitarse.



1.

Limitaciones Coste y Diseño Profundo

Coste

Este proceso conlleva un coste inmediato en tiempo y recursos que debe ser considerado una inversión y no un gasto perdido. Sin embargo la refactorización tiene un límite y es que no puede corregir errores de diseño profundo o fallos arquitectónicos graves.

Diseño

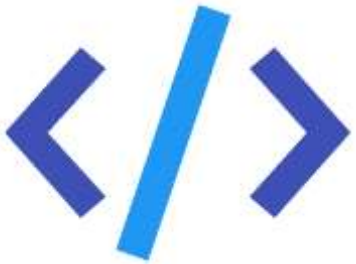
Si la base del sistema está mal concebida desde el principio la refactorización superficial no será suficiente y podría ser necesario reescribir partes completas del software.

1.

Patrones Comunes Renombrado y Extracción

Renombrado

El patrón de renombrar variables y métodos busca que el código sea autodocumentado de forma que el nombre explique claramente la intención y el uso sin necesidad de comentarios adicionales.



Extracción

Por otro lado la extracción de métodos es una técnica vital que consiste en tomar fragmentos de código de una función larga y moverlos a un nuevo método independiente. Esto reduce la complejidad ciclomática y mejora la cohesión haciendo que cada función realice una única tarea específica.

1.

Patrones Comunes Organización y Lógica

Mover

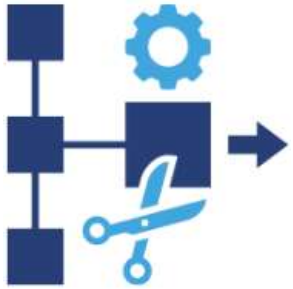
Mover métodos o clases es un patrón necesario cuando se detecta que una funcionalidad está en el lugar incorrecto de la arquitectura violando principios de responsabilidad.

Eliminar Duplicado

Eliminar código duplicado es obligatorio para cumplir el principio DRY y evitar tener que corregir el mismo error en múltiples sitios.

Simplificación

La simplificación de condicionales busca eliminar el anidamiento excesivo de sentencias if y else utilizando cláusulas de guarda para hacer el flujo de ejecución más lineal y comprensible.



1. Relación con las Pruebas

Las pruebas automatizadas constituyen el requisito indispensable para refactorizar con seguridad profesional. Actúan como un contrato que garantiza que el comportamiento del sistema se mantiene inalterado tras los cambios internos. Las pruebas unitarias son el principal apoyo en este proceso ya que permiten verificar instantáneamente si una modificación ha roto algo. El ciclo ideal de trabajo consiste en tener los tests en verde refactorizar el código y confirmar que los tests siguen en verde.

1.

Herramientas de Automatización IDE

Los entornos de desarrollo integrados modernos como NetBeans Eclipse IntelliJ o Visual Studio Code han revolucionado esta práctica ofreciendo herramientas automatizadas de refactorización. Estos IDEs no solo reemplazan texto sino que entienden la estructura sintáctica del código lo que permite realizar cambios complejos como extracciones o renombramientos masivos con total seguridad y precisión. Su uso minimiza drásticamente el error humano y acelera el proceso de limpieza del código.

2. Control de Versiones

- Desarrollos colectivos
- Herramientas de control de versiones
- Características principales
- Estructura
- Operaciones con clientes de control de versiones

2.

Trabajo Colaborativo y Control

El desarrollo de software actual es inherentemente colaborativo y requiere que múltiples personas trabajen simultáneamente sobre los mismos archivos. Los sistemas de control de versiones son la herramienta fundamental que gestiona estos cambios concurrentes evitando que el trabajo de un programador sobrescriba el de otro.

Además proporcionan una trazabilidad completa registrando qué autor realizó cada modificación exacta en qué fecha y con qué motivo permitiendo revertir el proyecto a cualquier estado anterior seguro en caso de error.

2.

Herramientas y Gestión de Proyectos

Git se ha establecido como el estándar industrial gracias a su arquitectura distribuida que otorga a cada desarrollador una copia completa del historial del proyecto permitiendo trabajar sin conexión y gestionar ramas de forma eficiente.

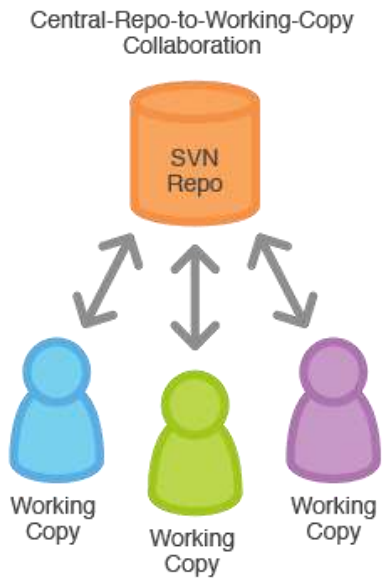
Aunque existen alternativas centralizadas como SVN o distribuidas como Mercurial la flexibilidad de Git para fusionar código y aislar nuevas funcionalidades en ramas independientes lo hace indispensable en proyectos reales para garantizar una integración continua y ordenada.

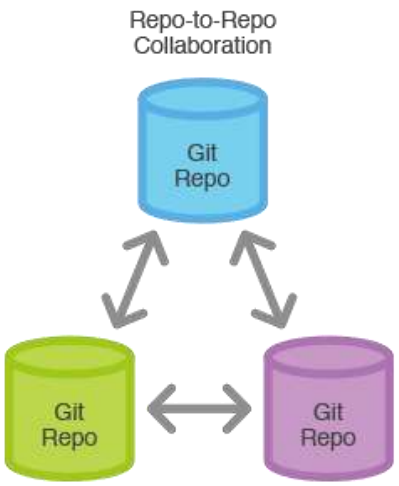
2.

Control de versiones

El control de versiones es una herramienta que permite gestionar los cambios realizados en un proyecto a lo largo del tiempo.

Facilita el trabajo colaborativo, evita la pérdida de información y permite recuperar versiones anteriores cuando sea necesario.



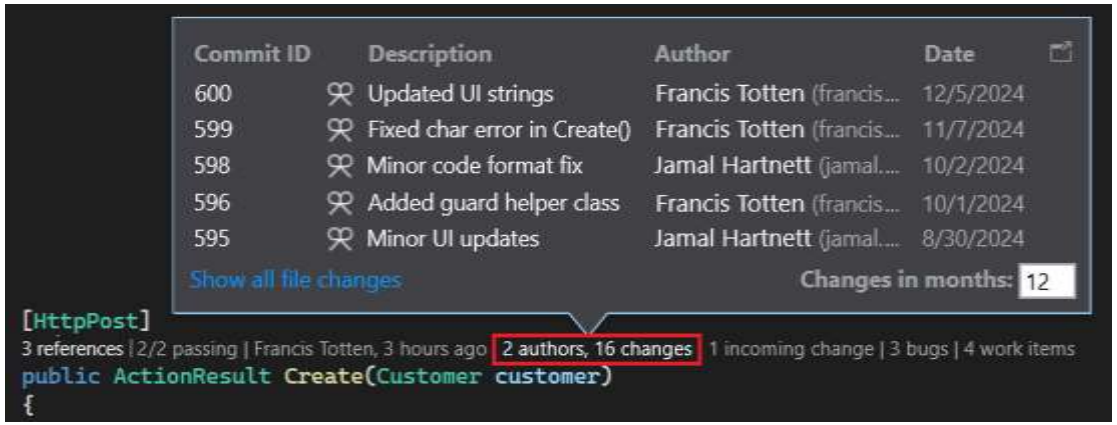


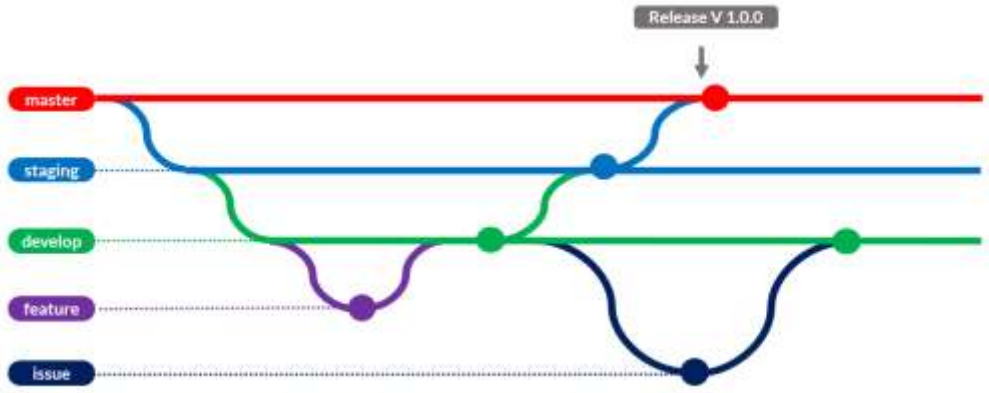
2.

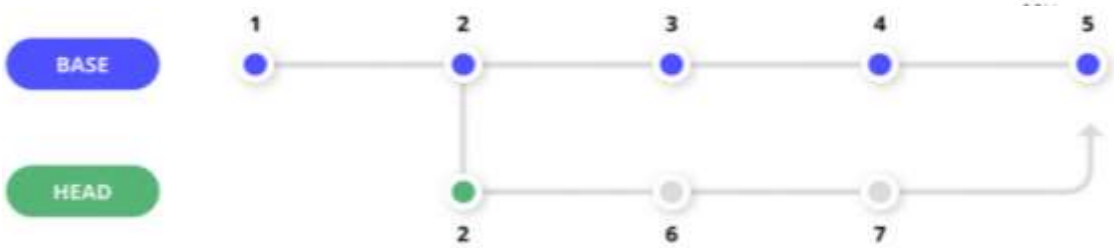
Características principales

Las principales características del control de versiones incluyen el historial de versiones, que registra todos los cambios realizados, el trabajo con ramas para desarrollar funcionalidades en paralelo.

Y las fusiones que permiten unir los cambios de distintas ramas.





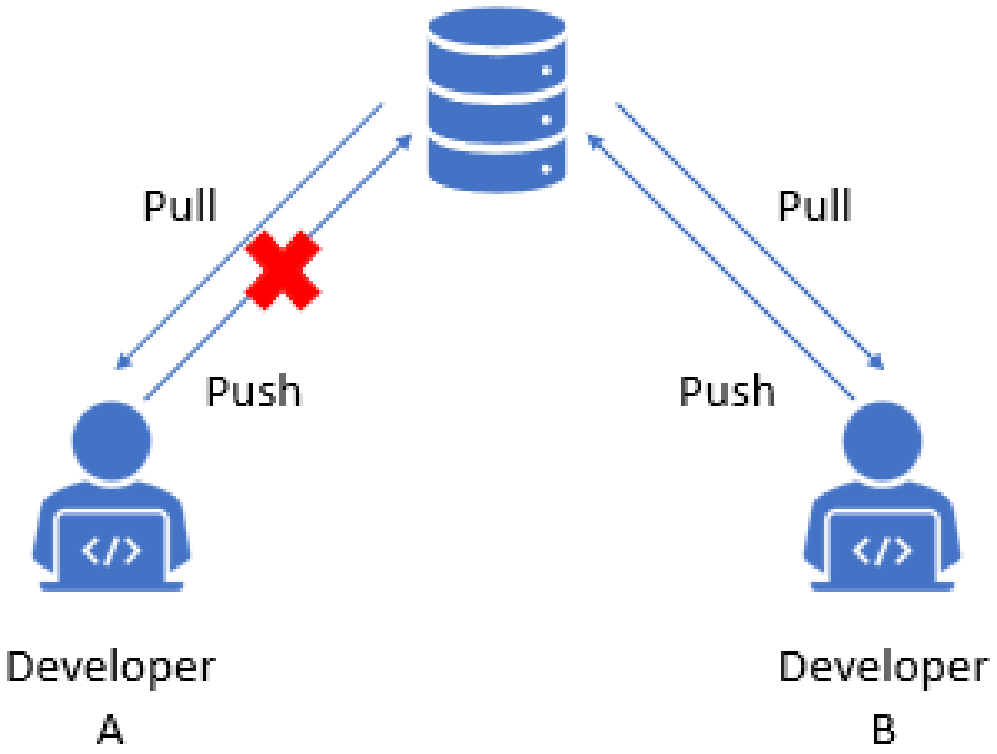


2.

Resolución de conflictos

La resolución de conflictos se produce cuando dos o más usuarios modifican la misma parte de un archivo.

El sistema detecta el conflicto y el usuario debe decidir qué cambios conservar o combinar para mantener la coherencia del proyecto.



2.

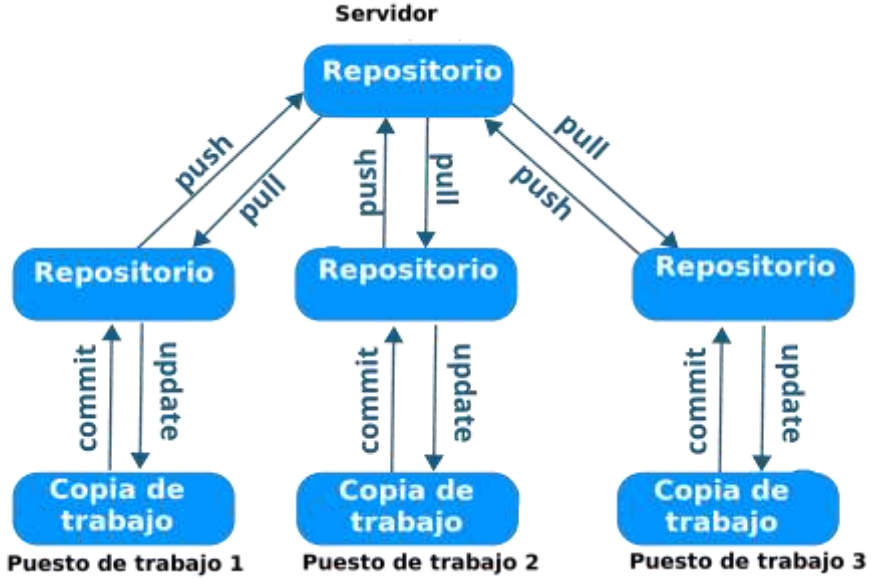
Estructura del sistema

El sistema de control de versiones se basa en un modelo cliente/servidor.

Cuenta con un repositorio remoto que almacena la versión principal del proyecto y copias locales que permiten a cada usuario trabajar de forma independiente.

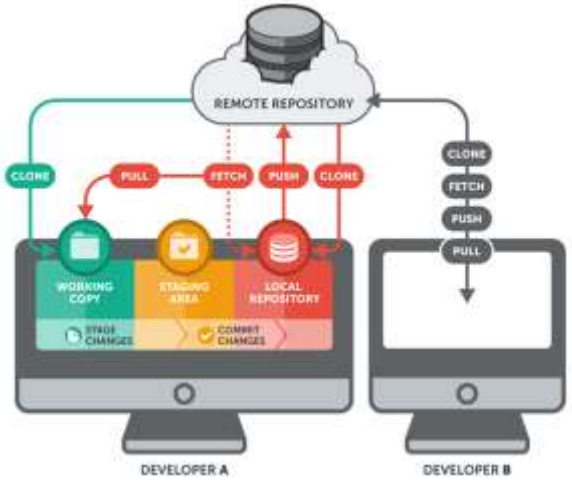
Control de versiones centralizado

Control de versiones distribuidos

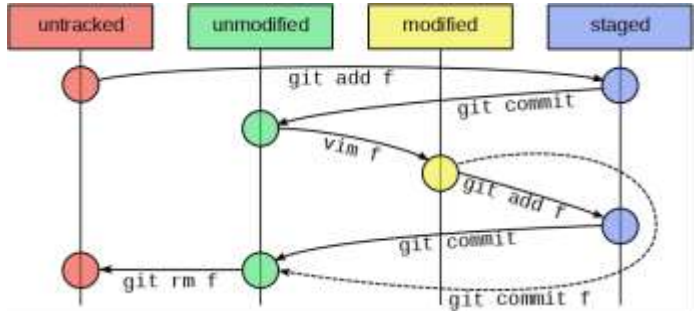


2.

Operaciones con clientes de control de versiones I



La descarga inicial del proyecto consiste en obtener por primera vez el repositorio remoto en el equipo local. A partir de ahí, la modificación de ficheros se lleva a cabo para añadir nuevas funcionalidades o corregir errores existentes.

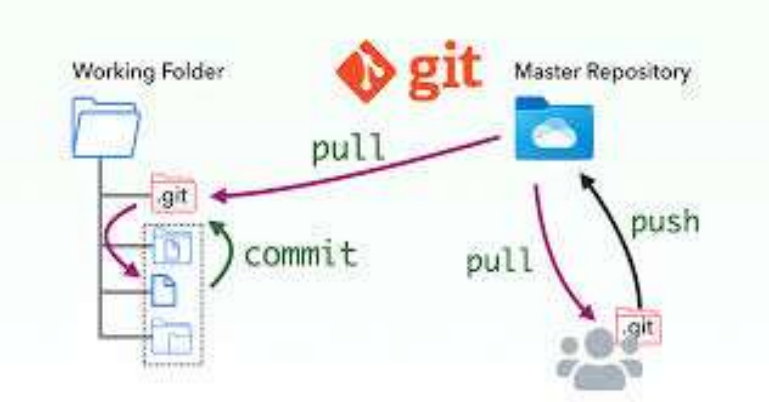


La actualización de los ficheros en local permite mantener el proyecto sincronizado con los cambios más recientes del repositorio remoto

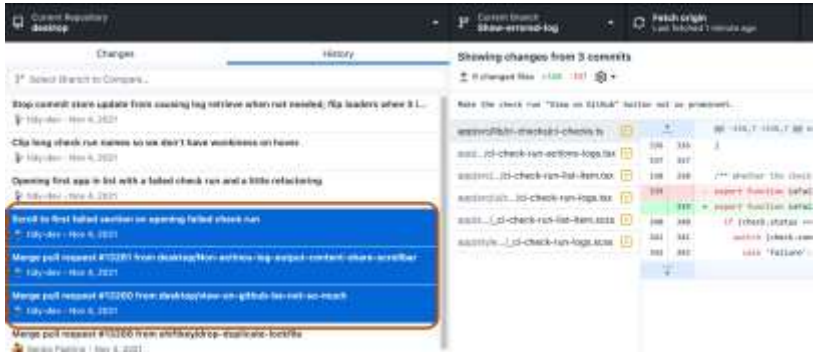
La subida de cambios al repositorio guarda las modificaciones realizadas en el entorno local y las pone a disposición del resto del equipo de trabajo.

2. Operaciones con clientes de control de versiones II

Los sistemas de control de versiones permiten la comparación entre versiones para analizar diferencias entre archivos.



También facilitan la restauración de versiones anteriores en caso de errores. La resolución de conflictos es necesaria cuando varios usuarios modifican el mismo archivo al mismo tiempo.



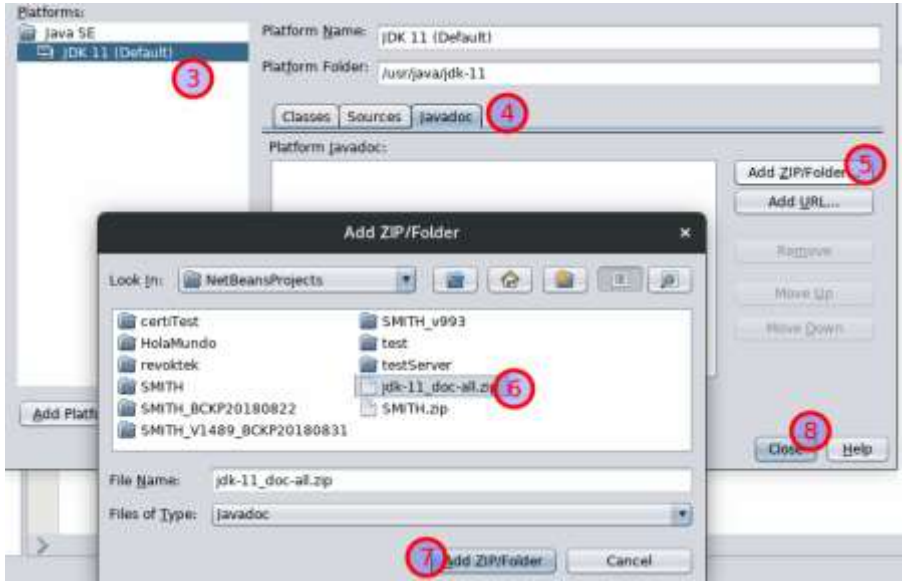
La consulta del historial permite revisar los cambios realizados a lo largo del

Documentación del software

Uso de comentarios
Herramientas de documentación automática
Alternativas de documentación

3.

Documentación del software y uso de comentarios



La documentación del software es un elemento clave en el desarrollo de sistemas informáticos, ya que permite comprender, mantener y evolucionar un programa a lo largo del tiempo.

Facilita el trabajo colaborativo y asegura la continuidad del proyecto.

Los comentarios son anotaciones incluidas en el código fuente que sirven para explicar su

3.

Finalidad y buenas prácticas de los comentarios

La finalidad principal de los comentarios es mejorar la legibilidad del código, reducir errores y facilitar su mantenimiento. Permiten comprender decisiones de diseño y partes complejas del programa.

Comentarios claros y concisos, justificación de decisiones

3.

Herramientas de documentación automática

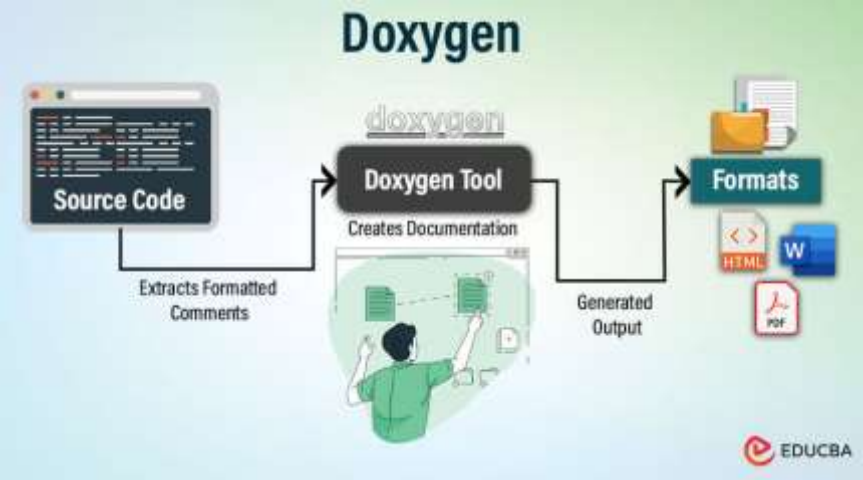
Las herramientas de documentación automática permiten generar documentación estructurada a partir del código fuente, utilizando comentarios específicos. Esto garantiza una documentación coherente y fácil de mantener.

```python
def function_name(parameters):
    """
    Function description

    Parameters:
    - parameter1: description of the parameter
    - parameter2: description of the parameter

    Returns:
    - return_type: description of the return type
    """
    # Function body
```

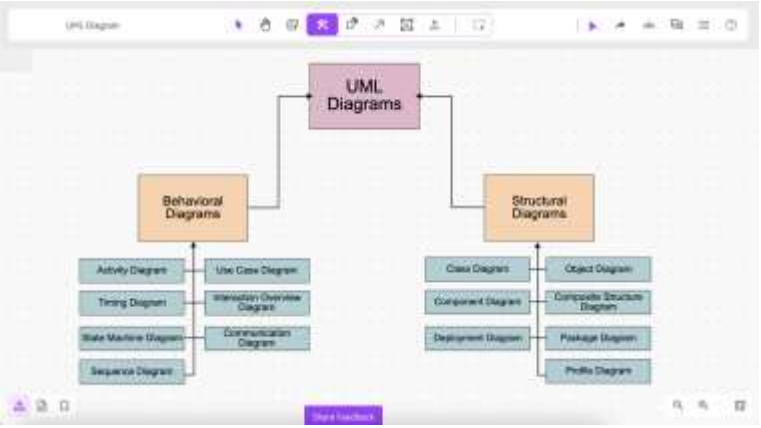
Entre las herramientas más utilizadas se encuentran JavaDoc, empleada en proyectos Java; Doxygen, compatible con múltiples lenguajes de programación; y los docstrings, utilizados en Python para documentar funciones, clases y módulos.

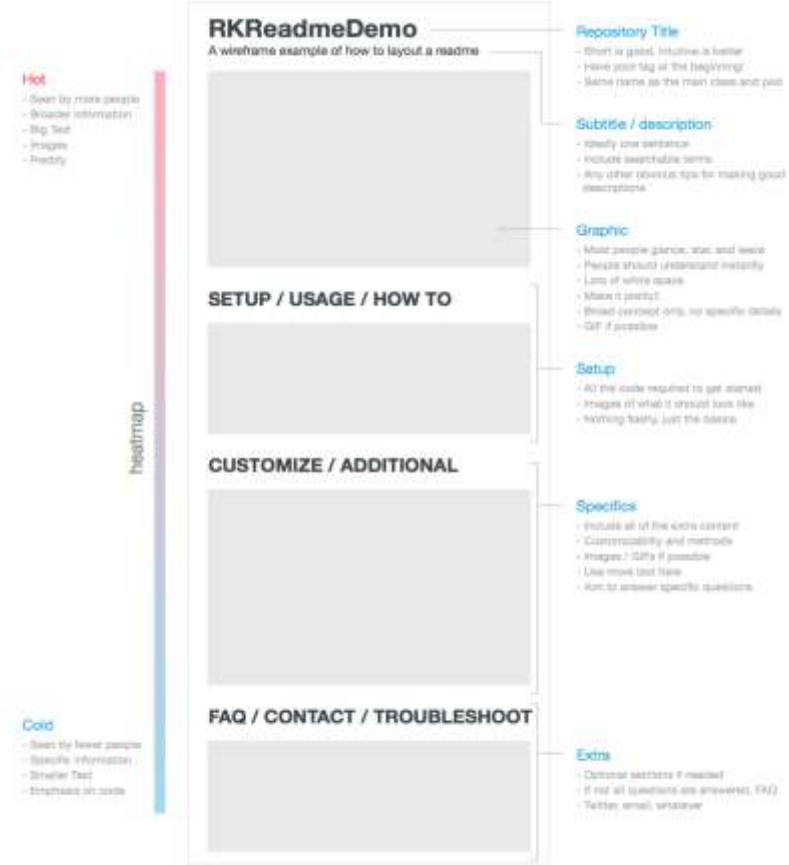


3.

Alternativas de documentación

Las wikis de proyecto permiten una gestión colaborativa del conocimiento, el archivo README proporciona una visión general del sistema, los manuales técnicos ofrecen información detallada y los diagramas UML permiten representar visualmente la estructura y el comportamiento del software.





Segunda Parte

Preguntas de tipo test

P

1. ¿Cuál de las siguientes opciones es una característica principal de un sistema de control de versiones?
   a) Almacenamiento en la nube exclusivamente
   b) Historial de versiones
   c) Edición de documentos en tiempo real
   d) Compresión automática de archivos

2. ¿Qué elementos forman parte de la estructura de un sistema de control de versiones?
   a) Servidor local, archivos temporales y caché
   b) Modelo cliente/servidor, repositorio remoto y copias locales
   c) Editor de código, depurador y compilador
   d) Usuario administrador y permisos avanzados

P

3. ¿Qué operación permite guardar las modificaciones locales y compartirlas con el resto del equipo?
   a) Descarga inicial del proyecto
   b) Actualización de ficheros en local
   c) Comparación entre versiones
   d) Subida de cambios al repositorio

4. ¿Cuál es la característica principal de la refactorización?

a) Corregir errores (bugs) y añadir nuevas funcionalidades al sistema

b) Reescribir completamente el software para solucionar fallos arquitectónicos graves

c) Alterar la estructura interna del código sin cambiar su comportamiento externo observable

d) Optimizar el código para que el usuario final note una

P

5. ¿Qué elemento se describe como el requisito indispensable para refactorizar con seguridad profesional? a) Utilizar un entorno de desarrollo (IDE) moderno como IntelliJ o Eclipse

b) Contar con una red de pruebas automatizadas

c) Obtener la aprobación del cliente antes de modificar el código

d) Eliminar todos los comentarios del código antes de empezar